Name :- Sana Munir Alam
Class :- BCS-4E
Roll# :- 24K-0573

DB Assignment -2

Date:

Question 1

A)
```
SELECT DISTINCT c.customer_name FROM Customers c
JOIN Purchase p ON c.customer_id = p.customer_id
JOIN Purchase_Details pd ON p.purchase_id = pd.purchase_id
JOIN Books b ON pd.book_id = b.book_id
WHERE b.price > (
        SELECT AVG(price) FROM Books)
);
```

B)
```
SELECT b1.title FROM Books b1
WHERE b1.price > (
    SELECT AVG(b2.price) FROM Books b2
    WHERE b2.publisher = b1.publisher );
```

C)
```
SELECT c.customer_name FROM Customers c
WHERE EXISTS (
    SELECT 1 FROM Purchases p
    JOIN Purchase_Details pd ON p.purchase_id=pd.purchase_id
    WHERE p.customer_id = c.customer_id
)
AND NOT EXISTS)
    SELECT 1 FROM Purchases p
    JOIN Purchase_Details pd ON p.purchase_id = pd.purchase_id
    JOIN Books b ON pd.book_id = b.book_id
    WHERE p.customer_id= c.customer_id AND b.publisher = 'Pearson'
);
```

Date:

Explanation :-

Exists ensure that the customer purchased at least one book
NOT Exists ensures the customer "never" purchased" a book published by 'Pearson'

D) SELECT title FROM Books b
WHERE NOT EXISTS (
SELECT 1
FROM Purchase_Details pd
WHERE pd.book_id = b.book_id
);

E) SELECT DISTINCT a.author_name FROM Authors a
JOIN Book_Authors ba ON a.author_id = ba.author_id
JOIN Books b ON ba.book_id = b.book_id
Where b.price > (
SELECT AVG(price) FROM Books
);

Explanation:- Calculate avg price of the whole books in system then find authors whose respective book price is greater than average we just calculated.

Date:

F) Select c.Customer_ID (
Select SUM(pd.quantity) FROM Purchase_Details pd
WHERE pd.purchase_id IN (
SELECT p.purchas_id FROM Purchases p
WHERE p.customer_id = c.customer_id
)
) AS Total_Qty FROM Customers c
WHERE (
SELECT SUM (pd.quantity) FROM Purchase_Detaids pd
WHERE pd.purchase_id IN (
SELECT p.purchase_id FROM Purchases p
WHERE p.customer_id = c.customer_id
)
) > 10 ;

Question 2

A) SELECT p.passenger_name, b.bus_name, r.source_city, r.destination_city
FROM Passengers p
JOIN Tickets t ON p.passenger_id = t.passenger_id
JOIN Buses b ON t.bus_id = b.bus_id
JOIN Routes r ON t.route_id = r.route_id ;

explanation :- Inner join used because we only want passengers who have an actual booking, matched with their assigned bus and route

B) SELECT p.passenger_name, t.travel_date, t.fare FROM Passengers p
JOIN Tickets t ON p.passenger_id = t.passenger_id;

explanation :- Inner join is used to fetch only those passengers who have atleast one ticket booked, along with their fare and travel date.

C) SELECT b.bus_name, COUNT(t.passenger_id) AS total_passengers
FROM Buses b
LEFT JOIN Tickets t ON b.bus_id = t.bus_id
GROUP BY b.bus_name ;

Date:

Explanation :- Left join is used so that all buses appear in the result, even those with no booking yet, their count will show 0.

D) SELECT r.source_city, r.destination_city, p.passenger_name
FROM Tickets t
JOIN Passengers p ON t.passenger_id = p.passenger_id
JOIN Routes r ON t.route_id = r.route_id ;

Explanation :- Inner join is used because we only need routes and passengers that are linked through an actual booked ticket.

E) CREATE VIEW PassengerTravelSummary AS
SELECT p.passenger_name, b.bus_name, r.source_city, r.destination_city,
E.travel_date
FROM Tickets t
JOIN Passengers p ON t.passenger_id = p.passenger_id
JOIN Buses b ON t.bus_id = b.bus_id
JOIN Routes r ON t.route_id = r.route_id ;

F) Part 1
ALTER TABLE Passengers ADD contact_number VARCHAR(15);

Part 2
ALTER TABLE Tickets MODIFY fare DECIMAL(10,2);

Part 3
ALTER TABLE BUSES DROP COLUMN capacity ;

Date:

Part 4

DROP TABLE Routes ;

Note:- Before dropping routes, we must either drop Tickets table first or remove the foreign-key constraint on route_id in Tickets, other-wise the Data base will throw a foreign key violation error. (referential integrity violated).

Question 3

A) 1- Workout Programs:

Program ID → Primary Key
Program Name
Duration
Difficulty Level
Coordinator Name

2- Trainers

Trainer ID → Primary Key
Trainer Name
Contact Number

3- Fitness Centers

Center ID → Primary Key
Center Name

Date:

B) 1- Program ⟷ Trainer

Relationship : Conducts / Assignet To

Cardinality : M : N (Many to Many)

Reason :

- One program can have multiple trainers
- One trainer can conduct multiple programs

2- Workout Program ⟷ Fitness Center

Relationship : Offered at

Cardinality : M : N (Many to Many)

Reason :

- One program can be offered at multiple centers
- One center can host multiple programs.

C) Linking table (links workout program and trainers)

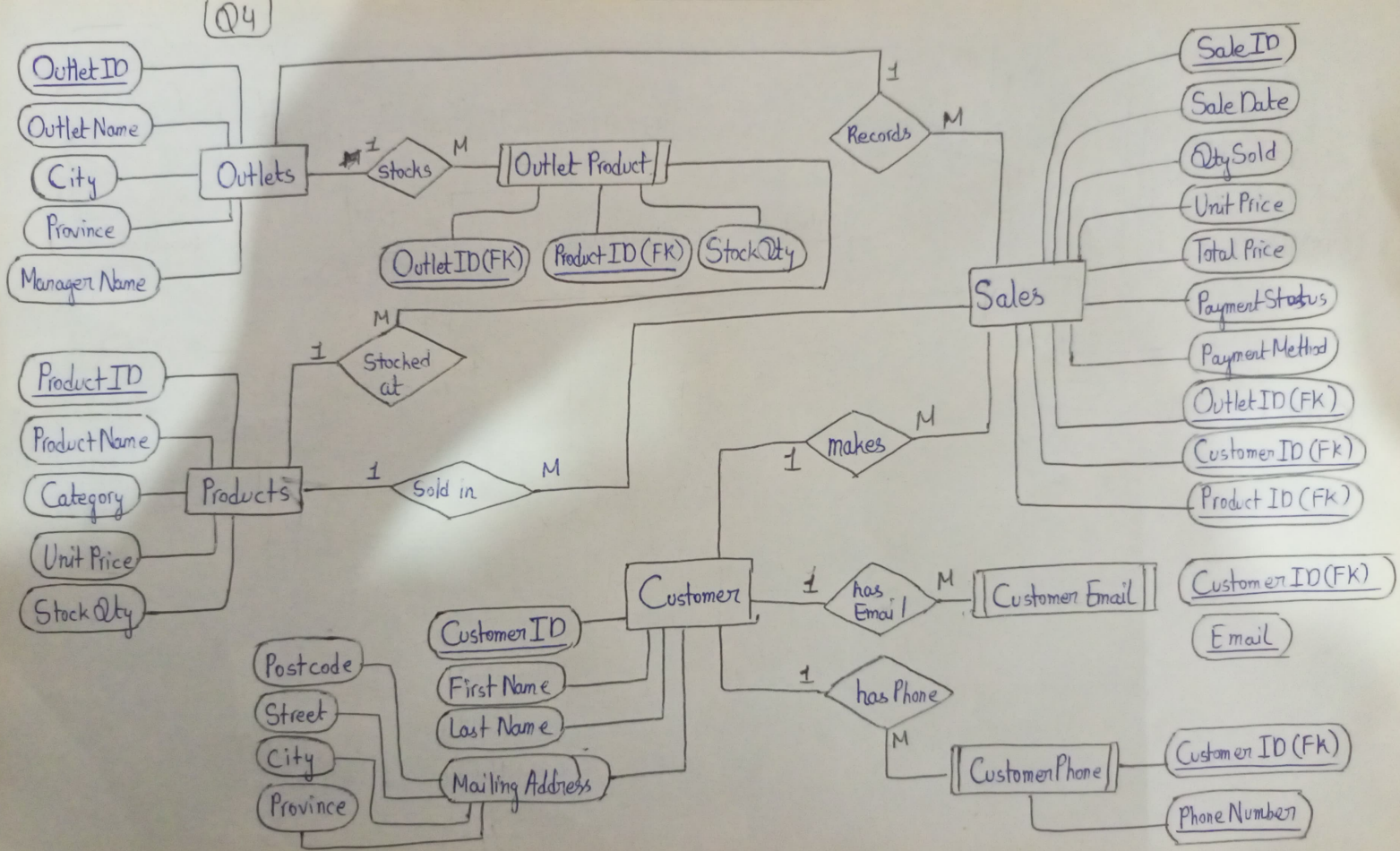
Program-Trainers (table name → Associative entity)

Program ID	Foreign Key (Workout Program) + PK
Trainer ID	Foreign Key (Trainers) + PK
Role	eg: Lead / Assitant

The composite primary key is (Program ID + Trainer ID) together as the same trainer can appear in multiple programs, and the same programs can have multiple trainers, but the combination must be unique.

Date:

D) Coordinator Name should not be modeled as a seperate entity because the system does not store any additional details about coordinators.
There are no relationship involving co-ordinator.
Each program has only one coordinator (so simple attribute)
Hence creating a seperate entity would unnecessarily complicate the design.


Q4
Outlet ID
Outlet Name
City
Province
Manager Name
Outlets
1
Stocks
M
Outlet Product
Outlet ID (FK)
Product ID (FK)
Stock Qty
1
Records
M
Sale ID
Sale Date
Qty Sold
Unit Price
Total Price
Payment Status
Payment Method
Outlet ID (FK)
Customer ID (FK)
Product ID (FK)
Sales
M
1
Stocked at
Product ID
Product Name
Category
Unit Price
Stock Qty
Products
1
Sold in
M
1
makes
M
Customer
1
has Email
M
Customer Email
Customer ID (FK)
Email
Customer ID
First Name
Last Name
Postcode
Street
City
Province
Mailing Address
1
has Phone
M
Customer Phone
Customer ID (FK)
Phone Number

Date:

Q5 Part(a)

Insertion Anamoly:

It occurs when certain data can not be inserted without the presence of other unrelated data.

In this case we can't add a new journalist unless they have submitted an article. A new fact-checking agency can not be recorded unless they have ~~submitted~~ verified at least one article. A new AI Model can not be added unless it is linked to an existing article.

Impact :- This restricts flexibility and forces incomplete or unnecessary data Insertion

Update Anamoly:

If an agency changes it's phone number, you must update it in every single row where that agency appears. If even one row is missed, the database becomes inconsistent (two rows show the same Agency ID but different phone numbers).

Deletion Anamoly:

If the only article verified by a particular agency is deleted, all information about that agency (Agency ID, Agency Name, Agency Phone) is permanently lost. The agency disappears from the system just because one article was removed.

Date:

Part b)

Assumption Made:

One article is submitted by exactly one journalist
One article is published on exactly one platform
A verification is uniquely identified by Verification ID
Each AI model has a unique name and version combination.

ArticleID → ArticleTitle, SubmissionDate, JournalistID, PlatformID
JournalistID → JournalistName, JournalistEmail
PlatformID → PlatformName
AIModelID → AIModelName, ModelVersion
AgencyID → AgencyName, AgencyPhone
VerificationID → VerificationDate, VerificationOutcome, ConfidenceScore, AgencyID, ArticleID

Composite key for Article-AIModel Analysis:
(ArticleID, AIModelID) → forms the linking table.

Part c) Step 1 ⇒ First Normal Form.
The table is already in 1NF assuming each cell holds one value.
The candidate key would be:
ArticleID, AIModelID, VerificationID.

Step 2: Second Normal Form
Remove all partial dependencies. So table looks like after 2NF applied

Date:

Table	Attributes
• Articles	Article ID, Article Title, Submission Date, Journalist ID, Platform ID,
• Journalists	Journalist ID, Journalist Name, Journalist Email
• Platforms	Platform ID, Platform Name
• AI Models	AI Model ID, AI Model Name, Model Version
• Agencies	Agency ID, Agency Name, Agency Phone
• Verifications	Verification ID, Verification Date, Verification Outcome, Confidence Score, Agency ID, Article ID
• Article AI Model	Article ID, AI Model ID } Linking Table

3rd Normalisation Form

Already in 3NF form as no transitive dependencies.

Article		Journalists	
- Article ID	PK	- Journalist ID	PK
- Article Title		- Journalist Name	
- Submission Date		- Journalist Email	Alternate Key (unique)
- Journalist ID	FK → Journalists		
- Platform ID	FK - Platforms		

Platforms		AI Models	
- Platform ID	PK	- AI Model ID	PK
- Platform Name		- AI Model Name	
		- AI Model Version	

Date:

Agencies
- Agency ID PK
- Agency Name
- Agency Phone

ArticleAIModel (linking table)
- Article ID PK+FK → Articles
- AIModelID PK+FK → AI models

Verifications
- Verification ID PK
- Verification Date
- Verification Outcome
- Confidence Score
- Article ID FK → articles
- Agency ID FK → Agencied

Part d) Data Redundancy Problem.

1- Agency Phone repeated across rows.
Type of anomaly : Update dnomaly
If an agency updates it's phone number, every verification row linked to that agency must be updated manually as one missed row = data in consistency.

2- AI Model Info repeated per article
Type of dnomaly : Update + Insertion
If a model version is corrected, it must be fixed in every article-analysis row, not just one place.

3- Repeated Journalist Information
dnomaly:- Update + redundancy

Date:

Journalist name and email repeated for every article hence redundancy.

4- Platform Name repeated per article.

Type of Anomaly :- Update anomaly

Renaming a platfor requires updating every article row tied to it.